

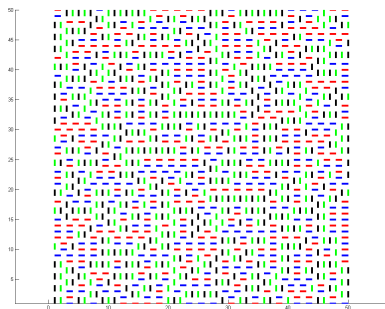
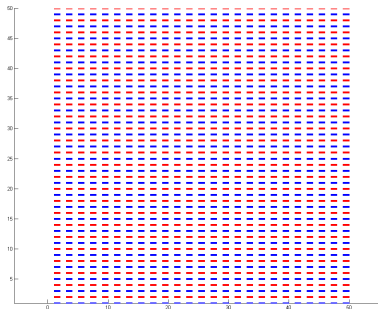
Monte Carlo Markov Chains

Computing Seminar with Undergraduate Researchers

Shannon Starr

University of Alabama at Birmingham

June 29, 2022



Abstract:

Imagine two different problems.

In the first one you have a desired probability distribution that you wish to sample from, such as uniform random permutations or a uniform random proper q -coloring of a graph.

In the second situation you have a cost function H on a finite (but large) set, and you wish to optimize H .

For both problems you may try to simulate using a Monte Carlo Markov chain.

In the latter case, you may use a Metropolis algorithm, in a Gibbs state at a particular temperature, with a schedule for cooling.

This is called simulated annealing.

Problem 1:

Suppose that you have a characterization of a given probability distribution on a finite set.

But you want to have a more detailed description in some way.

You want to see a typical sample.

(Or you want to repeatedly sample, in an IID manner, so as to generate some statistics for some feature of your distribution.)

Case 1: Uniform distributions

Suppose that $\mathcal{X}$ is a finite set, with $n \in \mathbb{N} = \{1, 2, \dots\}$,

$$n = |\mathcal{X}|.$$

Define a function

$$\mu : \mathcal{X} \rightarrow \mathbb{R},$$

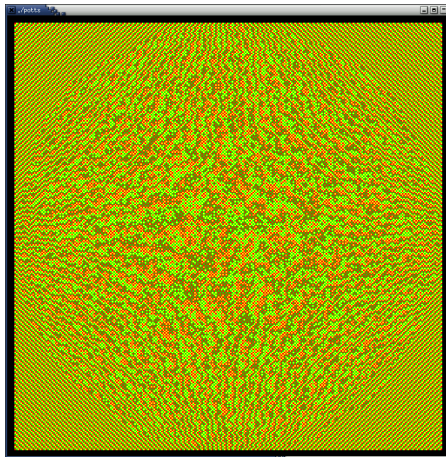
defined by

$$\forall x \in \mathcal{X}, \text{ we have } \mu(x) = \frac{1}{n}.$$

In reality, we might not know exactly what the number n is.

For example, we might be presented with $\mathcal{X}$ as a non-empty subset of a simpler finite set $\mathcal{Y}$. Then $n \leq |\mathcal{Y}|$.

Example: proper 3-colorings of square lattice



This picture is from Joakim Linde's website, based on a paper he co-wrote with Cris Moore. For that picture they used "coupling from the past." They also had special boundary conditions.

A finite graph is a pair $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where

- ▶ $\mathcal{V}$ is a finite set of vertices,
- ▶ $\mathcal{E}$ is the set of edges: it is any subset of the set of all pairs $\{v, w\}$ for $v, w \in \mathcal{V}$ and $v \neq w$.

As an example:

$$\mathcal{V}_L = \{\mathbf{v} = (v_1, v_2) : v_1, v_2 \in \mathbb{N} \text{ and } v_1, v_2 \in [1, L]\},$$

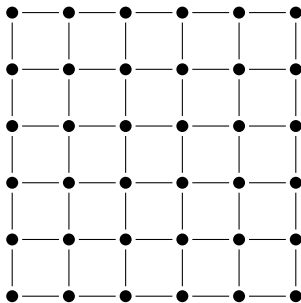
and, using the usual Euclidean magnitude/norm for vectors,

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2},$$

the edge set is the set of all pairs at distance 1 from each other

$$\mathcal{E}_L = \{\{\mathbf{v}, \mathbf{w}\} : \mathbf{v}, \mathbf{w} \in \mathcal{V}_L \text{ and } \|\mathbf{v} - \mathbf{w}\| = 1\}.$$

Trivia question: for the cardinalities $|\mathcal{V}_L|$ and $|\mathcal{E}_L|$ what are the formulas as functions of L ?



Given q colors for some $q \in \mathbb{N}$, a

PROPER q COLORING OF $\mathcal{G}$,

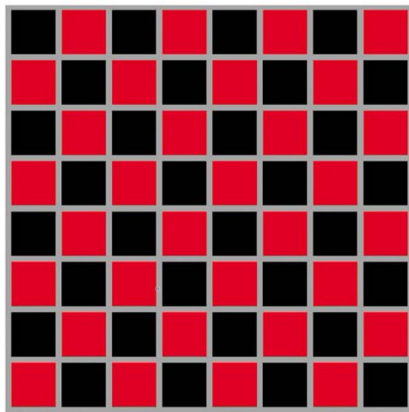
is a function

$$f : \mathcal{V} \rightarrow \{1, \dots, q\},$$

such that

$$\forall \{v, w\} \in \mathcal{E}, \text{ we have } f(v) \neq f(w).$$

The minimum q for the square lattice such that the set of proper q -colorings is non-trivial and “connected” is $q = 3$.



For $\mathcal{X}$ equal to the set of proper 3-colorings of the square lattice we know $|\mathcal{X}| \leq 3^{L^2}$.

Trivia fact: Elliott Lieb (and H.N.V. Temperley) showed the correct base is $(4/3)^{3/2} \approx 1.5396007$.

Monte-Carlo algorithm for this problem:

- Start with a known proper 3-coloring, such as the red-black checkerboard pattern. (In what follows $q = 3$.)
- Run a for- or while-loop. At each step:
 - Choose a vertex $v \in \mathcal{V}_L$, uniformly at random.
 - Choose a color $k \in \{1, \dots, q\}$, uniformly at random.
 - If the new mapping f' is a proper q -coloring, keep it.
 - Otherwise, revert to the previous proper q -coloring, f .
- Keep on running until you believe you have mixed up, or until you exhaust your patience and want to stop.

If you wait long enough the probability for your random proper q -coloring, generated this way, will approach μ , the desired probability distribution.

Symmetry: first instance of reversibility

Why does this work?

Suppose that (f, f') is a pair of proper q -colorings such that you can possibly go from f to f' in one step.

Then we have the important property:

$$\mathbf{Prob}(f_{t+1} = f' \mid f_t = f) = \mathbf{Prob}(f_{t+1} = f \mid f_t = f'),$$

where $f_0, f_1, \dots$ is the random sample of the chain of proper q -colorings you generate following your Monte Carlo algorithm.

To go from f to f' , you must randomly select the 1 vertex $v \in \mathcal{V}$ where $f(v) \neq f'(v)$, and then randomly select the 1 color $k \in \{1, \dots, q\}$ which is $f'(v)$:

$$\mathbf{Prob}(f_{t+1} = f' \mid f_t = f) = \frac{1}{q|\mathcal{V}|} = \mathbf{Prob}(f_{t+1} = f \mid f_t = f').$$

Matlab codes for a dimer shuffle

dominoMaker.m

```
halfsize=5;
size=2*halfsize;
NumDom=2*halfsize^2;
MtchngS = [];
for x=1:halfsize,
    for y=1:size,
        dom = [2*x-1,y,2*x,y];
        MtchngS = [MtchngS;dom];
    end
end
```

dominoPlotter.m

```
figure
hold on
for m=1:NumDom
    dom=MtchngS(m,:);
    plot([dom(1),dom(3)], [dom(2),dom(4)], 'k', 'LineWidth', 8)
end
```

dominoShuffle.m

```
Edge1 = [0,0,1,0];
Edge2 = [0,1,1,1];
Edge3 = [0,0,0,1];
Edge4 = [1,0,1,1];
dirRND = 1 + floor(2*rand);
if dirRND == 1,
    EdgeA = Edge1;
    EdgeB = Edge2;
    EdgeC = Edge3;
    EdgeD = Edge4;
else,
    EdgeA = Edge3;
    EdgeB = Edge4;
    EdgeC = Edge1;
    EdgeD = Edge2;
end
iRand = 1 + floor((size-1)*rand);
jRand = 1 + floor((size-1)*rand);
RowA = 0;
RowB = 0;
```

dominoShuffle.m (continued ...)

```
for m=1:NumDom,
    if MtchnoS(m,:) == [iRand,jRand,iRand,jRand]+EdgeA
        RowA=m;
    end
end
for m=1:NumDom,
    if MtchnoS(m,:) == [iRand,jRand,iRand,jRand]+EdgeB
        RowB=m;
    end
end
if RowA*RowB ~= 0,
    MtchnoS(RowA,:) = [iRand,jRand,iRand,jRand]+EdgeC;
    MtchnoS(RowB,:) = [iRand,jRand,iRand,jRand]+EdgeD;
end
```

More general reversibility

We may still be able to generate a Markov chain that drives the random sample towards the stationary distribution μ , even for non-uniform μ . The *Markov chain* is “reversible” relative to μ if

$$\forall x, y \in \mathcal{X}, \text{ we have } \mu(x)P(x, y) = \mu(y)P(y, x). \quad (*)$$

Here P is the Markov transition matrix:

$$\forall x, y \in \mathcal{V}, \text{ we have } P(x, y) = \mathbf{Prob}(X_{t+1} = y \mid X_t = x).$$

The probability distribution μ is a row-vector and multiplying on the left-hand-side of P :

$$(\mu \cdot P)(x) = \sum_{y \in \mathcal{X}} \mu(y)P(y, x) = \mu(x).$$

The last equality is true because of the reversibility condition (*) and because the row-sums for any Markov chain are all 1

$$\forall x \in \mathcal{X}, \text{ we have } \sum_{y \in \mathcal{X}} P(x, y) = 1.$$

Metropolis chains

There is a good textbook reference, free online, by David Levin

<https://pages.uoregon.edu/dlevin/>

whose coauthors are Yuval Peres and Elizabeth Wilmer.

A standard situation in statistical mechanics is that there is a cost function called the Hamiltonian

$$H : \mathcal{X} \rightarrow \mathbb{R},$$

and a number $\beta \in [0, \infty)$, called the inverse-temperature, and

$$\forall x \in \mathcal{V}, \text{ we have } \mu_\beta(x) = \frac{\exp(-\beta H(x))}{Z(\beta)},$$

where μ_β is called the Gibbs distribution and $Z(\beta)$ is a normalization constant called the partition function

$$Z(\beta) = \sum_{x \in \mathcal{X}} e^{-\beta H(x)}.$$

Now suppose that you have a symmetric Markov chain P on $\mathcal{X}$

$$\forall x, y \in \mathcal{V}, \text{ we have } P(x, y) = P(y, x).$$

Make a new Markov chain P_β as follows:

- Suppose X_t is the random state at time t .
- First consider a proposed move to a new random state $X_{t+1}^{(\text{prop})}$ determined by 1 step of the symmetric Markov transition matrix P .
- Then determine whether or not to accept the proposal or to revert to the starting state $X_{t+1} = X_t$.
 - If $H(X_{t+1}^{(\text{prop})}) \leq H(X_t)$ then accept: $X_{t+1} = X_{t+1}^{(\text{prop})}$.
 - If $H(X_{t+1}^{(\text{prop})}) > H(X_t)$ then you may still accept. Let

$$p = e^{-\beta H(X_{t+1}^{(\text{prop})})} / e^{-\beta H(X_t)},$$

be the probability of H for an unfair coin:

- if H accept, $X_{t+1} = X_{t+1}^{(\text{prop})}$,
- if T reject, $X_{t+1} = X_t$.

A generalization of the steepest descent/ascent algorithm

Here is a picture from Levin, Peres, Wilmer:

40 3. MARKOV CHAIN MONTE CARLO: METROPOLIS AND GLAUBER CHAINS

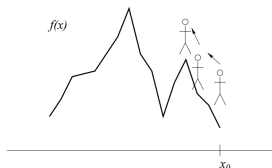


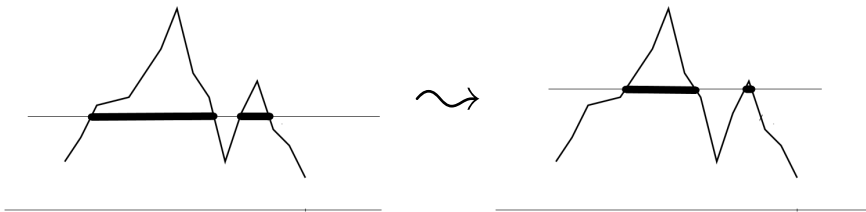
FIGURE 3.1. A hill climb algorithm may become trapped at a local maximum.

Usually for a complex cost function H on a vector space $\mathbb{R}^d$, you choose the direction $\mp \nabla H(\mathbf{x})$ and move in that direction a distance which is optimal, as calculated by 1 step of Newton's algorithm.

But if you hit a local min/max, the algorithm halts, and it may not be the global min/max.

Simulated annealing

If you choose a particular temperature, the Metropolis algorithm should equilibrate around temperature $T = 1/(k_B\beta)$ with average value of H (called the internal energy) and free energy monotonically related to temperature



If you slowly lower the temperature it should allow you to explore the actual minimum